

Server Scaling Predictor

End-to-End ML Pipeline for Automatic Server Scaling
in Persistent Multiplayer Games

Technical Write-Up | Game Infrastructure & MLOps

May 2026

*Production-Ready Time-Series Forecasting, Anomaly Detection,
and Automatic Infrastructure Scaling for Multiplayer Games*

1. Executive Summary

This project implements a comprehensive machine learning pipeline for predicting server load, detecting anomalous behavior, and enabling automatic infrastructure scaling in persistent multiplayer games. The system combines time-series forecasting (Prophet + LSTM), behavioral anomaly detection (Isolation Forest + domain rules), and a production-grade scaling executor with built-in safety mechanisms.

Built on synthetic data that realistically simulates hourly server telemetry (player concurrency, CPU, memory, tick rate, network), the pipeline demonstrates a complete MLOps lifecycle — from data generation through model training, evaluation, and interactive deployment. Primary outcomes include Prophet 7-day holdout MAE of **13.85 players** (MAPE 7.1%), LSTM next-hour MAE of **5.29 players** (early-stopped at epoch 40 of 100), and anomaly detection F1 of **0.71** on labeled synthetic anomalies.

The architecture is explicitly designed for production: it includes commented production code for automatic server provisioning via cloud APIs (Hetzner, AWS, Kubernetes) and runs safely in dry-run mode by default. This enables game development teams to monitor infrastructure health, optimize capacity planning, and implement cost-aware, revenue-gated scaling for long-running player-driven worlds. The dashboard ships with two data modes — **Reproducible** (fixed seed, identical results every run) and **Live Simulation** (random seed, fresh telemetry per session) — demonstrating the correct production pattern of training once and running inference continuously on new data.

2. Problem Statement & Business Value

Persistent multiplayer games face unique infrastructure challenges: player populations fluctuate dramatically due to content drops, viral events, and daily/weekly seasonality. Manual capacity planning cannot scale to thousands of concurrent users across multiple realms, nor can it respond quickly enough to sudden spikes (DDoS, viral launches) or drops (patch issues, mass churn).

Business Value: Industry benchmarks show that proactive capacity management can reduce infrastructure costs by 25–40% while improving player experience through reduced latency and queue times. This pipeline delivers (1) accurate load forecasting for just-in-time scaling, (2) anomaly detection to protect against exploits, and (3) a modular system where basic scaling is always available, while revenue-based realm expansion and auto-deployment are optional advanced features.

3. System Architecture

The pipeline follows a modular, production-oriented design with six core components:

- **Synthetic Data Generator** — Realistic hourly telemetry with daily/weekly seasonality, growth trends, and 3% injected anomalies (DDoS, botting, viral spikes, sudden drops).
- **Feature Engineering** — Cyclical encoding (sin/cos for hour/dow), lag/rolling features, and game-specific ratios (cpu_per_player, tick_rate efficiency, player_density).
- **Forecasting Ensemble** — Prophet for long-term planning with uncertainty intervals + PyTorch LSTM for short-term multivariate prediction.
- **Anomaly Detection** — Isolation Forest (200 trees) + rule-based boosters for high-precision detection of botting and DDoS patterns.
- **Recommendation Engine** — Cost-aware actions with priority tiers (CRITICAL / HIGH / MEDIUM) and estimated monthly costs.
- **Scaling Executor** — Production architecture (commented) for automatic Hetzner/AWS provisioning + safe dry-run mode for portfolio/demo use.

The system is designed so that synthetic data can be swapped for real Prometheus or custom game agent telemetry with minimal changes, and the scaling executor can be enabled for full automation once API credentials are configured.

4. Modeling Approach

4.1 Forecasting — Prophet + LSTM Ensemble

Prophet was selected for its strong performance on seasonal game telemetry and built-in uncertainty quantification. Key hyperparameters include multiplicative seasonality, `changepoint_prior_scale=0.05`, and added regressors (`is_weekend`, `hour_sin`, `dow_sin`). The model is trained on all data except the final 7 days, then evaluated on the holdout period.

PyTorch LSTM provides complementary short-term reactivity. A 2-layer LSTM with 64 hidden units processes 48-hour sequences of 10 engineered features to predict next-hour player load. Features are `StandardScaler`-normalised (fit on the training split only to prevent leakage) before being fed to the network — critical given the mixed magnitudes of inputs (e.g. `hour_sin`: `-1..1` vs `cpu_usage_percent`: `0..100`). Training uses MSE loss with Adam (`lr=0.001`), gradient clipping (`max-norm=1.0`) to prevent LSTM gradient explosion, `ReduceLROnPlateau` scheduling, and patience-based early stopping (`patience=8`) to restore the best validation checkpoint.

4.2 Anomaly Detection — Isolation Forest + Rules

Isolation Forest (`contamination=0.03`, 200 estimators) identifies multivariate outliers. This is augmented with game-specific rules that catch 100% of obvious cases with zero false positives: high CPU + low players (botting), sudden >40% player drop after high CPU (patch/churn event), and extreme network spikes (DDoS).

4.3 Scaling Executor — Production Architecture

The executor translates recommendations into infrastructure actions. In production it calls cloud APIs (Hetzner, AWS, or Kubernetes) to provision new shards or realms. For safety and portfolio use, it runs in dry-run mode by default and includes clear comments showing exactly which lines to uncomment when API tokens are available.

5. Evaluation & Results

The pipeline was evaluated on a 90-day synthetic dataset with realistic seasonality, growth trends, and 65 labeled anomalies. All models were trained with proper time-series holdout validation to prevent data leakage.

Metric	Value	Notes
Prophet MAE (7-day)	13.85 players	On <code>concurrent_players</code>
Prophet RMSE	18.18 players	
Prophet MAPE	7.1%	Excellent for ops planning
LSTM MAE (next hour)	5.29 players	Scaled inputs + target
LSTM RMSE	13.70 players	Early-stopped epoch 40/100
Anomaly Precision	0.71	On 64 injected anomalies
Anomaly Recall	0.72	Combined IF + rules: F1 0.71

What Do These Numbers Actually Mean?

MAE (Mean Absolute Error) of 13.85 players means that, on average, Prophet's 7-day forecast is off by about 14 players. Since player counts in this dataset range from roughly 80 to 300 concurrent, this is a small and operationally acceptable error. **MAPE (Mean Absolute Percentage Error)** of 7.1% means the forecast is typically within 7% of the actual number — industry standard for game server forecasting is under 15% MAPE. The LSTM MAE of 5.29 players on next-hour prediction reflects the benefit of proper feature + target normalisation (`StandardScaler`) and training regularisation (gradient clipping, early stopping, LR scheduling).

When Does the System Actually Recommend Scaling?

In simple terms: the system recommends adding a new server shard when it predicts that player load will stay above **82% of current capacity for 4 or more consecutive hours**. Opening entirely new realms is an ****optional advanced feature**** (disabled by default). When enabled, it can check revenue via Stripe API or a billing database before suggesting expensive new infrastructure. For anomalies, it flags suspicious patterns like sudden high CPU with low player engagement (possible botting) or extreme network spikes (possible DDoS). All critical actions require human approval; only routine scaling suggestions can be automated. Auto-deployment is also optional and requires admin authentication in production.

Feature importance analysis shows recency-derived features and `cpu_per_player` dominate both forecasting and anomaly models — validating the value of domain-specific feature engineering for game telemetry.

Cost & Revenue Projection Chart

The dashboard includes a 30-day Cost & Revenue Projection chart as an example of business-oriented visualization. The current implementation uses placeholder values (\$8 per 250 players for infrastructure and a default monthly revenue). In a production environment, these values would be dynamically pulled from your cloud provider's billing API (Hetzner, AWS, GCP) and your revenue system (Stripe, Paddle, or internal database). The projection logic is easily customizable and demonstrates how forecasting models can be extended beyond technical metrics into financial planning.

6. Dashboard & User Experience

The Streamlit dashboard provides six progressive-disclosure tabs, each serving a distinct audience need:

- **Trends** — Interactive multi-metric time-series plots with anomaly overlays. Sidebar controls for metric selection and history window.
- **Forecast** — Prophet 48h prediction with uncertainty bands + LSTM next-hour MAE callout. Actionable for pre-event capacity planning.
- **Anomalies** — Flagged rows table with anomaly type, score, and behavioral explanations for DDoS, botting, and churn patterns.
- **Recommendations** — Priority-coloured action cards (CRITICAL / HIGH / MEDIUM / LOW) with one-click dry-run Execute buttons and CSV export.
- **How It Works** — Plain-language pipeline walkthrough designed for stakeholder communication and portfolio/LinkedIn demos.
- **What-If Simulator** — Interactive sliders for current players, growth rate, and revenue that drive real-time scaling and realm recommendations.

Color coding (green / amber / red), help tooltips on every KPI, and a 30-day Cost & Revenue Projection chart lower cognitive load for ops teams. All actions execute in dry-run mode by default; auto-deployment requires an explicit toggle with admin authentication.

6.1 Reproducible Mode vs Live Simulation

A core design principle of the dashboard is the explicit separation between **model training** and **inference on new data** — the same separation that governs real production ML systems.

- **Reproducible Mode (default, seed=42)** — The synthetic generator produces an identical 2,160-row telemetry timeline on every run. All reported metrics (Prophet MAE 13.85, LSTM MAE 5.29, anomaly F1 0.71) are derived from this fixed dataset. Anyone who clones the repository and runs `python train.py` will reproduce the exact same artifacts and numbers, making portfolio claims fully verifiable.
- **Live Simulation Mode (random seed)** — A sidebar toggle switches the data layer to a randomly seeded generator. Each click of 'New Simulation' produces a statistically equivalent but numerically different timeline: different player peaks, different anomaly injection points, different CPU curves. The trained models (Prophet, LSTM, Isolation Forest) are not retrained — they continue running inference on the new data stream exactly as they would in production against live server telemetry. The active seed is displayed so any interesting simulation can be reproduced.

This distinction — train once on a representative dataset, then infer continuously on arriving data — is the fundamental operational pattern for deployed ML systems. Demonstrating it explicitly in the dashboard bridges the gap between a portfolio experiment and a production-ready architecture.

7. Limitations & Future Work

Current limitations include reliance on synthetic data (real telemetry integration is straightforward but requires game-specific agent development) and dry-run scaling mode (production API integration requires valid cloud credentials and master lobby endpoints). Future extensions include multi-realm hierarchical forecasting, online learning with River for post-launch drift detection, and direct integration with billing systems for true revenue-gated scaling decisions.

8. Code & Reproducibility

All code is released under the MIT license. Full reproducibility is guaranteed by pinned dependencies, fixed random seed (42), and serialized artifacts (Prophet .pkl, LSTM .pt with metadata.json containing exact metrics and training timestamp). The project follows the **train-once / infer-anywhere** pattern: python train.py produces immutable model artifacts that are then loaded by the dashboard, predict.py, or any downstream consumer. Switching the dashboard to Live Simulation mode exercises this separation directly — new synthetic telemetry flows through the same frozen models, producing fresh forecasts and recommendations without any retraining. One-command reproduction:

```
git clone https://github.com/yourusername/server-scaling-predictor.git
cd server-scaling-predictor && python -m venv venv && source venv/bin/activate
pip install -r requirements.txt && python train.py && streamlit run dashboard/app.py
```

This pipeline provides game development teams with a ready-to-deploy analytics backbone for intelligent, cost-aware server scaling — enabling sustainable operations for long-running, player-driven worlds.